

Kompilátor Datalogu s funkčnými symbolmi do relačnej algebry

Matúš Duchyňa

8. 6., 2026

Fakulta matematiky, fyziky a informatiky

Univerzita Komenského v Bratislave

Vedúci práce: doc. Mgr. Tomáš Plachetka, Dr.

- Experimentálny databázový systém s relačnou algebrou ako dotazovacím jazykom
- Relačná algebra
 - Umožňuje určiť vykonávací plán
 - Nízkoúrovňová
- Datalog ako nový dotazovací jazyk

Hlavný cieľ

Kompilátor z Datalogu do relačnej algebry

- Databázový systém môže obsahovať tabuľky
 - Vymenovaním prvkov
 - Popísané nejakým programom
- Kompilátor ako samostatný nástroj, bez závislosti na konkrétnom databázovom systéme
- Podpora funkčných symbolov

- Tabuľky - predikáty
- Vstavané tabuľky - vstavané predikáty (označené s \$ pred názvom)

Pod pojmom **parameter** rozumieme:

1. ľubovoľný term
2. ľubovoľný predikát alebo vstavaný predikát
3. ľubovoľný zoznam parametrov, ktorý zapisujeme ako $[P_1, P_2, \dots, P_n]$,
kde $P_1, P_2, \dots, P_n$ sú parametre

- Tabuľky - relácie
- Vstavané *jednoduché* tabuľky - vstavané relácie (označené s \$ pred názvom)
- Vstavané *zložité* tabuľky - vstavané operátory (označené s \$ pred názvom)

Klasická relačná algebra

- Množinové operátory, $\times$
- Π, σ, ρ
- $\bowtie, \triangleright, \bowtie_C$

Naša relačná algebra

- UNION
- JOIN
- ANTIJOIN
- REC

Parameter (v kontexte relačnej algebry) je:

1. ľubovoľný term
2. ľubovoľnú *reláciu* alebo vstavaný operátor (ktorého výstupom je relácia)
3. ľubovoľný zoznam parametrov, ktorý zapisujeme ako $[P_1, P_2, \dots, P_n]$, kde $P_1, P_2, \dots, P_n$ sú parametre

Operátor je $O(P_1, P_2, \dots, P_n)$, kde O je názov operátora a $P_1, P_2, \dots, P_n$ sú parametre operátora. Výstupom operátora je relácia.

Syntaktická analýza

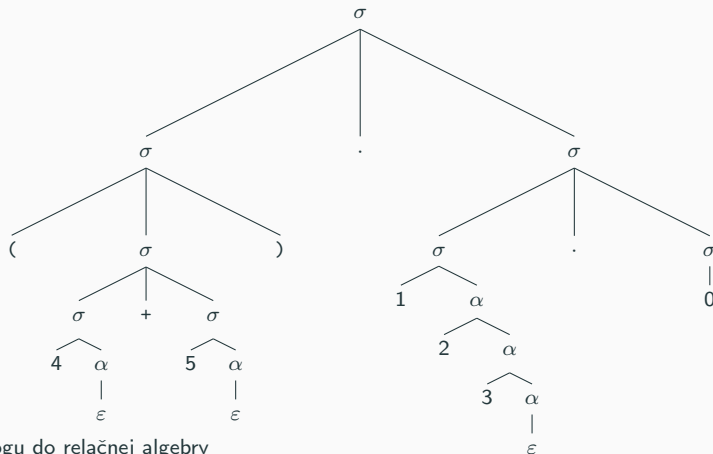
$G = (N, T, P, \sigma)$, kde

$N = \{\sigma, \alpha\}$,

$T = \{+, \cdot, (,), 0, \dots, 9\}$,

$P = \{\sigma \rightarrow (\sigma) \mid \sigma + \sigma \mid \sigma \cdot \sigma$
 $\mid 1\alpha \mid \dots \mid 9\alpha \mid 0,$
 $\alpha \rightarrow 0\alpha \mid \dots \mid 9\alpha \mid \varepsilon\}$

Odvodenie slova $(4 + 5) \cdot 123 \cdot 0$



$$\text{INT} = \{0\} \cup \{1, 2, 3, 4, 5, 6, 7, 8, 9\} \cdot \{1, 2, 3, 4, 5, 6, 7, 8, 9, 0\}^* \approx \mathbb{N}_0$$

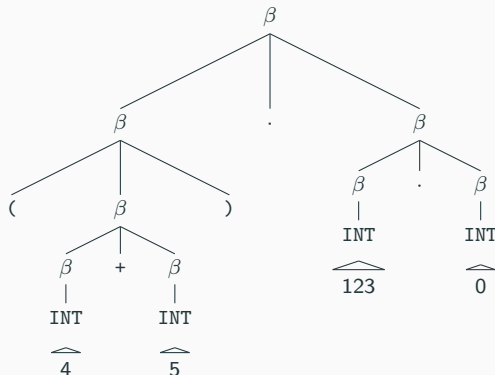
LPAREN((), INT(4), PLUS(+), INT(5), RPAREN()), MULT(·), INT(123), MULT(·), INT(0)

$G' = (N', T', P', \beta)$, kde

$N' = \{\beta\}$,

$T' = \{+, \cdot, (,), \text{INT}\}$,

$P' = \{\beta \rightarrow (\beta) \mid \beta + \beta \mid \beta \cdot \beta \mid \text{INT}\}$



Algoritmus prekladu

1. Vytvorenie lexeru a token streamu
2. Vytvorenie parseru a stromu odvodenia
3. Generácia AST z odvodenia
4. Preklad predikátov do relačnej algebry s nevyplnenými referenciami
 - 4.1 Preklad definícií pomocou **JOIN** a **UNION**
 - 4.2 Preklad klauzúl pomocou **JOIN** a **ANTIJOIN**
 - 4.3 Spojenie klauzúl do jednej relácie pomocou **UNION**
5. Zostavenie výsledného výrazu v relačnej algebre
 - 5.1 Zostavenie grafu závislosti predikátov
 - 5.2 Zabalenie rekurzívnych predikátov do **REC**
 - 5.3 Nahradenie referencií výrazmi, ktoré im prislúchajú
 - 5.4 Zabalenie výsledného výrazu do **JOIN**-u reprezentujúceho dotaz

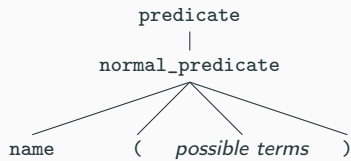
1. a 2. krok

ANTLR

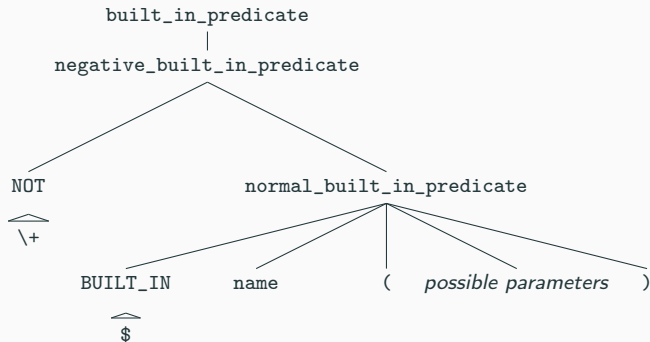
```
grammar datalog;
QUERY_MARKER: '?-';
BODY_HEAD_SEP: ':-';
NOT: '\+';
BUILT_IN: '$';
program: (clause | definition)* query;
query: QUERY_MARKER predicate EOL;
clause: name '(' ')' BODY_HEAD_SEP
        predicate '(' ',' predicate)* EOL
        | name '(' term '(' ',' term)* ')'
        BODY_HEAD_SEP predicate
        '(' ',' predicate)* EOL;
```

```
definition: name BODY_HEAD_SEP
            '{' term_tuple '(' ',' term_tuple)*
            '}' EOL;
term: variable
     | function '(' term '(' ',' term)* ')';
name: LOWER (LOWER | NUMBER | '_' )*;
function: LOWER (LOWER | NUMBER | '_' )*;
variable: UPPER (UPPER | NUMBER | '_' )*;
UPPER: [A-Z]+;
LOWER: [a-z]+;
NUMBER: [0-9]+;
WS: [ \t\r\n]+ -> skip
```

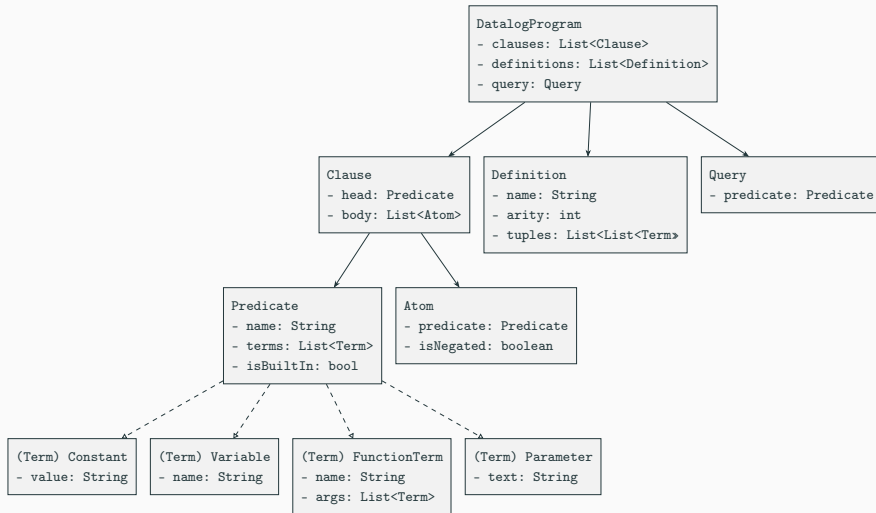
3. Generácia AST z odvoduenia



Obyčajný predikát



Negovaný vstavaný predikát



4. Preklad predikátov do relačnej algebry s nevyplnenými referenciami

1) Definície

`color :- {(r()), (b())}.`

sa preloží ako

```
UNION([
  JOIN([$true], [[]], [r()]),
  JOIN([$true], [[]], [b()])
])
```

2) Klausuly

`t(X, Z) :- e(X, Y), e(Y, Z),
 \+fn(Y), \+fe(X, Y).`

sa preloží ako

```
ANTIJOIN([
  JOIN([e, e],
    [[X, Y], [Y, Z]],
    [X, Y, Z]
  ),
  fn, fe],
  [[X, Y, Z], [Y], [X, Y]],
  [X, Z]
])
```

3) Zjednotenie výsledkov

`p(X) :- $a(X).
p(Y) :- $b(Y).`

sa preloží ako

```
UNION([
  JOIN([$a], [[X]], [X]),
  JOIN([$b], [[Y]], [Y])
])
```

2 sémanticky ekvivalentné programy:

```
p(Z) :- $do10(X), $do10(Y),  
        $plus(X, Y, Z).
```

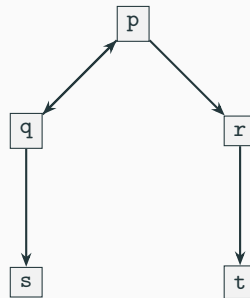
```
p(Z) :- $plus(X, Y, Z),  
        $do10(X), $do10(Y).
```

```
JOIN(  
    [$do10, $do10, $plus],  
    [[X], [Y], [X, Y, Z]],  
    [Z]  
)
```

```
JOIN(  
    [$plus, $do10, $do10],  
    [[X, Y, Z], [X], [Y]],  
    [Z]  
)
```

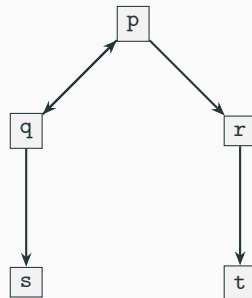
5. Zostavenie výsledného výrazu v relačnej algebre

- $p \rightarrow \text{JOIN}([q, r], [[X], [X]], [X])$
- $q \rightarrow \text{JOIN}([p, s], [[X], [X]], [X])$
- $r \rightarrow \text{JOIN}([t, t], [[X], [X]], [X])$
- $s \rightarrow \text{UNION}([\dots])$
- $t \rightarrow \text{UNION}([\dots])$



5. Zostavenie výsledného výrazu v relačnej algebre

- $p \rightarrow \text{REC}(p, \text{JOIN}([q, r], [[X], [X]], [X]))$
- $q \rightarrow \text{REC}(q, \text{JOIN}([p, s], [[X], [X]], [X]))$
- $r \rightarrow \text{JOIN}([t, t], [[X], [X]], [X])$
- $s \rightarrow \text{UNION}([\dots])$
- $t \rightarrow \text{UNION}([\dots])$



Zostavenie dotazu $?- q(f(X)).$

```
REC(q, JOIN([  
    p,  
    s  
], [[X], [X]], [X]))
```

Zostavenie dotazu $?- q(f(X)).$

```
REC(q, JOIN([  
    p,  
    UNION(...)]  
], [[X], [X]], [X]))
```

Zostavenie dotazu $?- q(f(X)).$

```
REC(q, JOIN([  
  REC(p, JOIN([  
    q,  
    r  
  ], [[X], [X]], [X])),  
  UNION(...]  
], [[X], [X]], [X]))
```

Zostavenie dotazu $?- q(f(X)).$

```
REC(q, JOIN([
  REC(p, JOIN([
    q,
    JOIN([
      t,
      t
    ]], [[X], [X]], [X])
  ], [[X], [X]], [X])),
UNION(...])
], [[X], [X]], [X]))
```

Zostavenie dotazu $?- q(f(X)).$

```
REC(q, JOIN([  
  REC(p, JOIN([  
    q,  
    JOIN([  
      UNION(...),  
      UNION(...)  
    ]), [[X], [X]], [X])  
  ], [[X], [X]], [X])),  
UNION(...)  
], [[X], [X]], [X]))
```

Zostavenie dotazu $\text{?- } q(f(X)).$

```
JOIN([  
  REC(q, JOIN([  
    REC(p, JOIN([  
      q,  
      JOIN([  
        UNION(...),  
        UNION(...)  
      ]), [[X], [X]], [X])  
    ], [[X], [X]], [X])),  
    UNION(...)  
  ], [[X], [X]], [X]))  
], [[f(X)], [f(X)]]
```

Ďakujem za pozornosť!